



S.G. Ganesh

The 'Broken Window' Theory

It is common to see software projects fail. One important cause is 'design and code rot'. In this article, let's try understanding the causes, in the light of a popular theory.

Development projects are often completed within tight deadlines to deliver working software, and hence managers focus only on externally visible product quality aspects, such as reliability, stability, performance, security, etc. Other less visible or not immediately measurable aspects, such as maintainability or reusability, are generally ignored. Most projects fail because they do not meet customer requirements (typically, non-functional requirements such as reliability); if the project survives, it moves on to the maintenance phase.

During maintenance, changes are made to fix or enhance features in the software without much focus on improving design or code quality. If such changes continue to be made, the design and code starts 'decaying'—the visible symptoms are known as 'code smells' in the refactoring community. If efforts to take up 'refactoring activities' are not made, it reaches a situation in which developers dread to touch the code. First, it becomes extremely difficult to understand the design and code; so any attempts to make even minor changes to the code could break the software! When the software becomes 'fragile', managers and customers wake up and try to do 'something' to get the situation under control. However, at this point, it is often too late to address the problem, and hence the project gets 'scrapped'. Soon, someone decides to use some other software, or write new software from scratch. For large enterprise software, the effort required for such 're-engineering' activities often costs millions of dollars.

Why does 'software decay' happen so quickly? Why don't developers follow good programming practices to keep the design and code clean? One way to explain this phenomenon is through the 'broken window' theory.

The 'broken window' theory was first introduced by Wilson and Kelling in 1982: "Consider a building with a few broken windows. If the windows are not

repaired, the tendency is for vandals to break a few more windows. Eventually, they may even break into the building, and if it's unoccupied, perhaps become squatters or light fires inside."

In India, this theory is easy to explain using the traffic jams that happen so very often. When a few vehicles break the rules and create confusion in the absence of traffic police, others, too, break the rules and make their own way through the traffic, which quickly leads to chaos!

In a software project, developers often do notice that the existing design and code is not clean, yet managers and leaders focus on 'getting the work done as soon as possible' rather than on 'getting it right'. Given the fact that the programming best practices have already been abandoned, there is no reluctance in breaking more rules, particularly when no one notices. This quickly leads to chaos, and the software becomes 'fragile', leading to scrapping the project.

A successful approach to the 'broken window' problem is to address the situation when things are under control, and when the problem is small. This is especially true for software. Things can go out of control very quickly in software. It is easier to do small refactorings with every fix or enhancement, than to get approvals for refactoring activities that require large budgets and a lot of time. It is understandable that managers can't usually get an approval to take up long-lead-time refactoring activities, but no one stops them from allocating a little extra time to ensure the quality of the code for every change done. Developers should be aware that breaking programming best practices is taken as a serious problem. To a great extent, these two approaches will keep the maintenance projects under control. 

By: S.G. Ganesh

The author works for Siemens (Corporate Technology). You can reach him at sgganesh@gmail.com.